Log Scrubber - User Guide

`log_scrubber.py` is a standalone command-line utility and importable Python library for scrubbing CUI, PII, PHI, credentials, and other sensitive data from Splunk field-value exports and log samples.

This guide is the CLI-focused quick-start. For the comprehensive project documentation covering both the CLI and the Paydirt browser tool, see the `README.md` at the repository root.

Compliance coverage: CMMC (CUI marking detection per NIST SP 800-171), HIPAA (PHI identifiers including SSN, Luhn-validated payment cards, NPI, phone), GDPR (personal data including IPs and device identifiers).

No dependencies beyond Python 3.9+ standard library.

Quick Start

```
log_scrubber/
├── log_scrubber.py
├── log_scrubbing_config.csv    <- place your config here
└── (your input files)
```

```
# Auto-detect mode from file contents (easiest)
python log_scrubber.py my_fields.csv

# Batch multiple files - mode auto-detected per file
python log_scrubber.py ~/Downloads/*.csv

# Explicit mode (still supported)
python log_scrubber.py fieldsummary my_fields.csv
python log_scrubber.py samples my_events.csv

# Include raw values for comparison
python log_scrubber.py fieldsummary my_fields.csv --include-raw

# Disable CUI marking detection (if your data isn't CUI-regulated)
python log_scrubber.py my_events.csv --no-cui
```

Output is written to `my_fields_scrubbed_20260420_103045.csv` (auto-timestamped).

Output Format

Fieldsummary Mode

By default, the output contains scrubbed values only - the raw values column is removed so the output is safe to share:

```
Field Name, Scrubbed Values, Count, Distinct Count
```

Use `--include-raw` to keep the original raw values alongside the scrubbed values (useful for reviewing what was changed):

```
Field Name, Raw Values, Scrubbed Values, Count, Distinct Count
```

Samples Mode

Log sample events are scrubbed in place - the output file contains only scrubbed events.

Step-by-Step Workflow

1. Discover Sourcetypes (optional)

If you don't already know which index and sourcetype to target, run this first against your Splunk metadata index:

```
| tstats count where index=* AND sourcetype=* earliest=-30d BY sourcetype, index,
source
| stats values(index) as indexes, values(source) as sources, sum(count) as
event_count by sourcetype
| sort sourcetype
```

As written it lists every sourcetype (`sourcetype=*`); narrow it to a specific sourcetype or wildcarded filter (e.g., `cisco:asa` or `cisco:*`) and adjust the `index=...` filter for your own environment. Keep the filter specific in large deployments or the search can run for a very long time. The `earliest=-30d` window also avoids surfacing sourcetypes that no longer exist.

Click **Export** -> choose **CSV** -> save the file using the convention `<company>_<environment>_sourcetype_discovery.csv`.

This lists each matching sourcetype with its indexes, sources, and event counts so you can pick a target for the next two searches. **Do not scrub this export** - sourcetype and index names need to stay accurate for downstream use (CIM macros, Data Refinery *Sourcetype CSV* entry, etc.). Only redact a name by hand if it itself contains sensitive content.

2. Export Field Values from Splunk Web

Run this SPL in Splunk Web (adjust index, sourcetype, and time range):

```
index=<your_index> sourcetype="<your_sourcetype>" earliest=-7d@d latest=now
| fieldsummary maxvals=15
| search field!="_*" AND field!="date_*" AND field!="linecount"
```

```
  AND field!="punct" AND field!="timestartpos" AND field!="timeendpos"
  AND field!="splunk_server_group"
```

Click **Export** -> choose **CSV** -> save the file using the convention `<company>_<environment>_<sourcetype>_fieldsummary.csv`.

3. Export Log Samples from Splunk Web

```
index=<your_index> sourcetype="<your_sourcetype>" earliest=-1d@d latest=now
| dedup punct | head 30
```

Click **Export** -> choose **CSV** -> save the file using the convention `<company>_<environment>_<sourcetype>_log-samples.csv`.

Tip: consider dropping the `| head 30` cap (but keeping `| dedup punct`) to capture *every* distinct event pattern. `dedup punct` already collapses the result set to one event per unique punctuation shape, so the unbounded query usually returns only a few dozen to a few hundred rows - completely manageable for most sourcetypes. The 30-event cap can silently hide infrequent-but-important patterns (rare error variants, privileged-user actions, edge-case payloads), so for small or medium sourcetypes the unbounded form is often the better choice. Keep the cap for very high-cardinality sourcetypes where the unbounded result would be too large.

Alternatively, copy/paste raw events into a plain `.txt` file (one event per line). The scrubber auto-detects the format.

4. Scrub the Exports

Skip the discovery export from Step 1 - it should pass through untouched. Run the scrubber on the field-value and log-sample exports:

```
# Auto-detection (recommended)
python log_scrubber.py guardduty_fields.csv
python log_scrubber.py guardduty_samples.csv

# Or scrub everything from today's Downloads in one shot
python log_scrubber.py ~/Downloads/*.csv
```

5. Review and Send

Check the `*_scrubbed_*` output files to verify sensitive data was replaced, then transmit the scrubbed files to the downstream recipient (LLM pipeline, email, ticket, etc.).

> ***Read every line before you send.*** *The scrubber redacts what it can recognize by shape (IPs, emails, tokens, SSNs, CUI markings) or by structured field name. It* ***cannot*** *detect bare personal names, free-text content, or organization-internal terms that have no distinguishing pattern. Common blind spots: an* `assigned_to` */* `assignee` */* `requester` */* `owner` *value like* `John Smith` *sitting in an unstructured* `_raw`

> *event, a `comment` / `description` / `notes` field containing customer names or case details, a person-named asset like `JSMITH-LAPTOP`, internal project or customer code names, branch and team names, and stack traces that echo user input. If a sensitive value lives in its own JSON key, CSV column, or fieldsummary field, you can target it with an `@json,<field_name>,<replacement>` rule and re-run - the `@json` shortcut covers all three. If it's buried in narrative text, no automated tool can find it reliably. Your eyes are the control of last resort.*

Configuration

Config File Location

The scrubber automatically looks for `log_scrubbing_config.csv` in these locations (first match wins):

1. Current working directory: `./log_scrubbing_config.csv`
2. Data subdirectory: `./data/log_scrubbing_config.csv`
3. Same directory as the script: `<script_dir>/log_scrubbing_config.csv`
4. Data subdirectory of script: `<script_dir>/data/log_scrubbing_config.csv`

The simplest approach is to place the config file in the same folder as `log_scrubber.py`. You can also specify a config explicitly:

```
python log_scrubber.py fieldsummary fields.csv --config /path/to/my_rules.csv
```

Built-in Scrubbing (Always Active)

Even without a config file, the scrubber applies these regex patterns automatically:

Pattern	**Replacement**
IPv4 addresses (`192.168.1.50`)	`10.0.0.x`
AWS ip- hostnames (`ip-10-50-26-117`)	`ip-10-0-0-x`
Email addresses (`admin@corp.com`)	`user@example.com`
FQDN hostnames (`server01.company.com`)	`host.example.com`
UNC paths (`\\server\share`)	`\\SERVER\SHARE`
Domain usernames (`CORP\jsmith`)	`DOMAIN\user`
MAC addresses (`00:1A:2B:3C:4D:5E`)	`00:00:00:00:00:00`
US Social Security Numbers (valid ranges only)	`000-00-0000`
Luhn-validated credit cards	`4111-1111-1111-1111`
US phone numbers	`555-555-5555`
Windows user SIDs (`S-1-5-21-*`)	`S-1-5-21-0-0-0-0`
AWS access keys (`AKIA...`, `ASIA...`)	`AKIAREDACTEDREDACTED`

Pattern	Replacement
JSON Web Tokens (`eyJ...`)	`eyJREDACTED.REDACTED.REDACTED`
GitHub PATs (`ghp_*`, `gho_*`, etc.)	`ghp_REDACTED`
Slack tokens (`xoxb-*`, `xoxp-*`, etc.)	`xoxb-REDACTED`
Stripe keys (`sk_live_*`, `sk_test_*`)	`sk_test_REDACTED`
Google API keys (`AIza*`)	`AIzaREDACTED`
PEM private key blocks	`[REDACTED_PRIVATE_KEY]`
HTTP Authorization headers	`Authorization: <scheme> REDACTED`
URL query-string credentials	`password=REDACTED`, etc.
US National Provider Identifiers (Luhn-validated)	`1234567893`

CUI markings (CMMC / NIST SP 800-171) trigger whole-line redaction with a placeholder that preserves the marking category and original byte count. See the README for the full list of detected marking styles.

Config File Format

The config file is a CSV with four types of rules:

Text substitution - replaces literal strings anywhere in the data:

```
# Simple replacement
sensitive-hostname,single,REDACTED_HOST

# Random replacement (picks one each time)
my-secret-domain.com,random,"example1.com,example2.com,example3.com"

# Two-column shorthand (implicit single mode)
my-company.com,example.com
```

JSON field rules (`@json` prefix) - targets specific field names at any nesting depth in JSON data:

```
@json,accessKeyId,REDACTED_KEY
@json,accountId,random,"000000000001,000000000002,000000000003"
@json,userName,REDACTED_USER

# Qualified path - matches "arn" ONLY when nested under userIdentity
@json,userIdentity.arn,arn:aws:iam::REDACTED:user/REDACTED
```

Key-value tag matching - handles AWS/Azure/GCP tag structures like `{"key": "Owner", "value": "admin@corp.com"}`:

```
# Use the tag's key name (not "key" itself)
@json,Owner,random,"user_a@example.com,user_b@example.com"
@json,Environment,REDACTED_ENV
```

Bulk name lists (`@names` prefix, v1.3.1+) - one config row expands at load time into one text rule per listed name, so an employee/contractor roster doesn't need a separate row per person. Supports both `single` and `random` replacement modes, with the same semantics as ordinary text rules (in `random` mode each occurrence picks independently, so a given source name may map to different replacements at different points in the file):

```
# Every listed name maps to the same replacement
@names,"Alice Smith,Bob Jones,Carol Davis",single,REDACTED_NAME

# Each occurrence picks randomly from the replacement pool
@names,"Thomas Kincade,Mary Bruce,Joe Wadamaker",random,"John Doe,Jane Doe,Joe
Smith"
```

The names list is just a quoted CSV field, so it can be broken across lines for readability:

```
@names,"Thomas Kincade,
Mary Bruce,
Joe Wadamaker",random,"John Doe,Jane Doe,Joe Smith"
```

Lines starting with `#` are treated as comments.

Example Config File

```
# === Text Substitution Rules ===
acme-corp.com,single,example.com
acmecorp,single,examplecorp
my-splunk-server,single,splunk-host

# === JSON Field Rules ===
@json,accessKeyId,REDACTED_KEY
@json,accountId,random,"000000000001,000000000002,000000000003"
@json,userName,REDACTED_USER
@json,sourceIPAddress,10.0.0.x
@json,Owner,random,"user_a@example.com,user_b@example.com"
@json,Name,REDACTED_NAME
@json,Environment,random,"dev,staging,prod"

# === Bulk Name Lists ===
@names,"Alice Smith,Bob Jones,Carol Davis",random,"John Doe,Jane Doe,Joe Smith"
```

Command Reference

```
usage: log_scrubber.py [-h] [--config CONFIG] [--output OUTPUT]
                       [--no-builtin] [--no-cui] [--include-raw]
                       [--dry-run] [--quiet] [--version]
                       [mode] input [input ...]
```

Argument	Description
`mode`	Optional. Either `fieldsummary` or `samples`. Omit for auto-detection.
`input`	One or more input file paths. Glob patterns (`*.csv`) are expanded.
`--config`, `-c`	Path to scrubbing config CSV (auto-detected if not specified)
`--output`, `-o`	Output file path (single input only; ignored in batch mode). Default: `<input>_scrubbed_<timestamp>.<ext>`
`--no-builtin`	Disable built-in regex patterns (IPs, emails, tokens, SSN, CC, etc.)
`--no-cui`	Disable CUI marking detection (enabled by default)
`--include-raw`	Include the original raw values column in fieldsummary output (default: scrubbed only)
`--dry-run`	Show what would be done without writing output
`--quiet`, `-q`	Suppress informational output
`--version`	Print version and exit

Supported Input Formats

Fieldsummary Mode

Accepts two CSV formats (auto-detected):

1. **Splunk fieldsummary export** - columns: `field`, `count`, `distinct_count`, `values`, etc.
2. **Pre-scrubbed export** - columns: `Field Name`, `Raw Values`, `Scrubbed Values`, `Count`, `Distinct Count`. This is the same format the scrubber itself writes with `--include-raw`, so re-running the tool on its own output is supported.

Both formats are handled automatically. Output column headers are normalized to `Field Name` and `Scrubbed Values` regardless of input format.

Samples Mode

Handles three formats (auto-detected):

1. **CSV with `_raw` column** - standard Splunk CSV export. The `_raw` column is scrubbed in place.
2. **Plain text** - one event per line (syslog, key=value, etc.)
3. **JSON / JSONL** - single-line or multi-line JSON events. Brace depth tracking handles multi-line JSON objects that span multiple lines.

Library Use

`log_scrubber.py` is import-safe - the CLI banner and argument parsing are guarded by `if __name__ == "__main__"`, so `import log_scrubber` from another module has no side effects.

Class API (recommended for new code):

```
from log_scrubber import Scrubber

scrubber = Scrubber(config_path='log_scrubbing_config.csv')
safe_text = scrubber.scrub(raw_log_line)

# Batch convenience
safe_batch = scrubber.scrub_many([line1, line2, line3])

# Programmatic construction with inline rules
scrubber = Scrubber(
    text_rules=[('acme.com', 'single', 'example.com')],
    json_field_rules=[('accountId', 'single', '000000000000'),
                      ('userIdentity.arn', 'single',
'arn:aws:iam::REDACTED:user/REDACTED')],
    enable_builtins=True,
    enable_cui=True,
)
```

A single `Scrubber` instance is safe to share across threads as long as the rule lists aren't mutated after construction.

Functional API (backward compatible):

```
from log_scrubber import parse_scrubbing_config, scrub_text

text_rules, json_rules = parse_scrubbing_config('log_scrubbing_config.csv')
clean = scrub_text(raw, text_rules, json_rules)

# Optional flags (default True) for v1.1+ feature control
clean = scrub_text(raw, text_rules, json_rules,
                   enable_builtins=True, enable_cui=True)
```

Existing callers that don't pass the new kwargs get the v1 behavior plus the new scrubbing layers automatically.

Tips

Reuse configs across tools. The same `log_scrubbing_config.csv` works with both `log_scrubber.py` and the Paydirt browser tool. If you maintain a config file for one, the other will use it as-is.

Review output before sending. Automated scrubbing handles known patterns, but always spot-check the output for any environment-specific data the rules might have missed.

Large JSON events (GuardDuty, CloudTrail, etc.) are handled natively - the scrubber parses JSON at any nesting depth and applies `@json` rules recursively, including AWS/Azure/GCP tag structures.

Add rules incrementally. Start with the built-in patterns, review the output, then add `@json` and text rules to the config for environment-specific identifiers that slipped through.

Try the demo. The `paydirt_demo.log` file in the project root exercises every scrubbing category. Run it through the scrubber to see all the built-in patterns in action:

```
python log_scrubber.py paydirt_demo.log
```

Version History

For the full version history, see the Version History section in the project README.

License

`log_scrubber.py` is licensed under the Apache License, Version 2.0. See the `LICENSE` and `NOTICE` files in the repository root for the full terms.

Machine Data Insights Inc. *"There's Gold In That Data!"*® | machinedatainsights.com

Version 1.3.3 - November 8, 2026